

CKA: Application Lifecycle Management

KodeKloud Complete Study Notes — Certified Kubernetes Administrator

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

Application Lifecycle Management Section

Introduction

This article covers essential aspects of application lifecycle management, including rolling updates, rollbacks, configuration methods, scaling strategies, and self-healing design fundamentals.

Hello, my name is Mumshad Mannambeth, and welcome to this lesson on application lifecycle management. In this article, we cover essential aspects such as rolling updates and rollbacks, various application configuration methods, scaling strategies, and the fundamentals of self-healing application design.



Many of these topics are covered in depth throughout the course. If you are already familiar with some of this material, feel free to skip ahead or review the practice tests to reinforce your knowledge.

We will begin by exploring the concepts of rolling updates and rollbacks as part of a comprehensive overview of application lifecycle management. This approach ensures that you grasp both the theoretical aspects and practical implementations.

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/lms.txt> Use this file to discover all available pages before exploring further.

Rolling Updates and Rollbacks

This guide covers managing updates and rollbacks in Kubernetes deployments, including rollouts, versioning, deployment strategies, and practical commands for minimal downtime.

Welcome to this guide on managing updates and rollbacks in Kubernetes deployments. In this article, we explore key concepts such as rollouts, versioning, and various deployment strategies. We also provide practical commands to update your deployments with minimal downtime and to revert changes when necessary.

This article covers the process of monitoring deployment rollouts, updating container images, and performing rollbacks using Kubernetes commands.

Understanding Rollouts and Versioning

When you create a deployment, Kubernetes initiates a rollout that establishes the first deployment revision (revision one). Later, when you update your application—say by changing the container image version—Kubernetes triggers another rollout, creating a new revision (revision two). These revisions help you track changes and enable rollbacks to previous versions if issues arise.

Rollout and Versioning



Revision 1



nginx:1.7.0



nginx:1.7.0



nginx:1.7.0



nginx:1.7.0



nginx:1.7.0



nginx:1.7.0



nginx:1.7.0



nginx:1.7.0



nginx:1.7.0

Revision 2



nginx:1.7.1



nginx:1.7.1



nginx:1.7.1



nginx:1.7.1



nginx:1.7.1



nginx:1.7.1



nginx:1.7.1



nginx:1.7.1



nginx:1.7.1

To monitor and review these rollouts, you can use the following commands:

Check the rollout status:

```
```bash theme={null} kubectl rollout status deployment/myapp-deployment
```

View the history of rollouts:

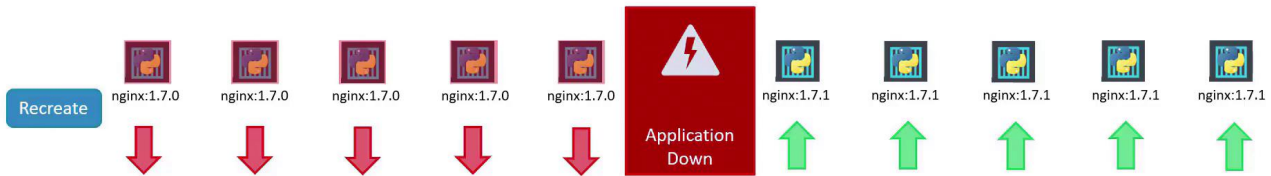
```
```bash theme={null}
kubectl rollout history deployment/myapp-deployment
```

Deployment Strategies

There are different strategies to update your applications. For example, consider a scenario where your web application is running five replicas.

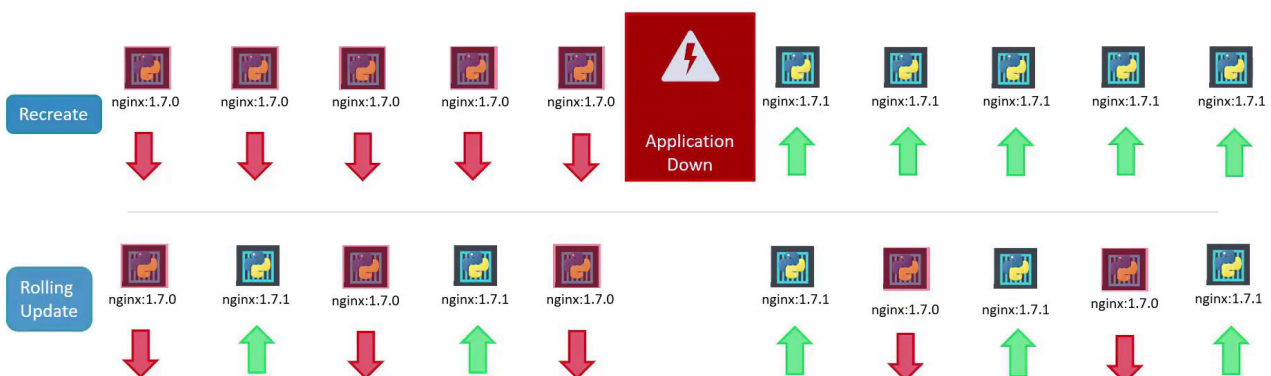
One approach is the "recreate" strategy, which involves shutting down all existing instances before deploying new ones. However, this method results in temporary downtime as the application becomes inaccessible during the update.

Deployment Strategy



A more seamless approach is the "rolling update" strategy. Here, instances are updated one at a time, ensuring continuous application availability throughout the process.

Deployment Strategy



If no strategy is specified when creating a deployment, Kubernetes uses the rolling update strategy by default.

Updating a Deployment

There are several methods to update your deployment, such as adjusting the container image version, modifying labels, or changing the replica count. A common practice is to update your deployment definition file and then apply the changes.

For example, consider the following deployment definition:

```
```yaml theme={null} apiVersion: apps/v1 kind: Deployment metadata: name: myapp-deployment labels: app: myapp type: front-end spec: replicas: 3 selector: matchLabels: type: front-end template: metadata: name: myapp-pod labels: app: myapp type: front-end spec: containers: - name: nginx-container image: nginx:1.7.1
```

After updating the file, apply the changes:

```
```bash theme={null}
kubectl apply -f deployment-definition.yml
```

This action triggers a new rollout and creates a new deployment revision.

Alternatively, you can update the container image directly using the following command:

```
```bash theme={null} kubectl set image deployment/myapp-deployment nginx-container=nginx:1.9.1
```

<Callout icon="lightbulb" color="#1CB2FE">

Remember, using `kubectl set image` updates the running deployment but does not modify your deployment definition file.

</Callout>

## Viewing Deployment Details

To retrieve detailed information about your deployment—including rollout strategy, scaling events, and more—use the following command:

```
```bash theme={null}
kubectl describe deployment myapp-deployment
```

This output shows different details depending on the strategy used:

- **Recreate Strategy:** Events indicate that the old ReplicaSet is scaled down to zero before scaling up the new ReplicaSet.
- **Rolling Update Strategy:** The old ReplicaSet is gradually scaled down while the new ReplicaSet scales up.

For example, a deployment with the recreate strategy might display the following events:

```
```bash theme={null} Name: myapp-deployment Namespace: default CreationTimestamp: Sat, 03 Mar 2018
17:01:55 +0000 Labels: app=myapp Annotations: deployment.kubernetes.io/revision=2 kubectl.kubernetes.io/
change-cause=kubectl apply --filename=deployment-definition.yml Selector: 5 desired, 1 updated, 5 total, 5
available, 0 unavailable StrategyType: Recreate MinReadySeconds: 0 Pod Template: Labels: app=myapp
type=front-end Containers: nginx-container: Image: nginx:1.7.1 Port: Conditions: Type Status Reason ----
----- Available True MinimumReplicasAvailable Progressing True NewReplicaSetAvailable OldReplicaSets:
NewReplicaSet: myapp-deployment-54c7d6ccc (5/5 replicas created) Events:
Type Reason Age From Message -----
Normal ScalingReplicaSet 11m deployment-controller Scaled up replica set myapp-deployment-6795844b58 to 5 Normal ScalingReplicaSet 11m deployment-controller
```

Scaled down replica set myapp-deployment-6795844b58 to 0 Normal ScalingReplicaSet 56s deployment-controller Scaled up replica set myapp-deployment-54c7d6ccc to 5

In contrast, a rolling update strategy output would reflect gradual scaling changes:

```
```bash theme={null}
kubectl describe deployment myapp-deployment
```

```

`bash theme={null} Name: myapp-deployment Namespace: default CreationTimestamp: Sat, 03 Mar 2018
17:16:53 +0800 Labels: app=myapp Annotations: deployment.kubernetes.io/revision=2 kubectl.kubernetes.io/
change-cause=kubectl apply --filename=deployment-definition.yml Selector: 6 desired, 5 updated, 6 total, 4
available, 2 unavailable StrategyType: RollingUpdate MinReadySeconds: 0 RollingUpdateStrategy: 25% max
unavailable, 25% max surge Pod Template: Labels: app=myapp type=front-end Containers: nginx-container:
Image: nginx Port: Conditions: Type Status Reason ---- -
Available True MinimumReplicasAvailable
Progressing True ReplicaSetUpdated OldReplicaSets: myapp-deployment-67c749c58c (1/1 replicas created)
NewReplicaSet: myapp-deployment-75d7bdbd8d (5/5 replicas created) Events:
Type Reason Age From Message -----
Normal ScalingReplicaSet 1m deployment-controller
Scaled up replica set myapp-deployment-67c749c58c to 5
Normal ScalingReplicaSet 1m deployment-controller
Scaled down replica set myapp-deployment-75d7bdbd8d to 2
Normal ScalingReplicaSet 1m deployment-controller
Scaled up replica set myapp-deployment-67c749c58c to 4
Normal ScalingReplicaSet 1m deployment-controller
Scaled down replica set myapp-deployment-75d7bdbd8d to 3
Normal ScalingReplicaSet 0s deployment-controller
Scaled down replica set myapp-deployment-75d7bdbd8d to 1
Normal ScalingReplicaSet 0s deployment-controller
Scaled down replica set myapp-deployment-67c749c58c to 0

```

Upgrading and Rolling Back

During an upgrade, Kubernetes creates a new ReplicaSet for the updated containers while the original ReplicaSet continues to run.

<Frame>

```
! [The image illustrates a Kubernetes deployment with two replica sets, each containing multiple pods, labeled 'v1' and 'v2'.]
</Frame>
```

If an issue is detected after an upgrade, you can revert to the previous version using the rollback feature.

```
``bash theme={null}
kubectl rollout undo deployment/myapp-deployment
```

This command scales down the new ReplicaSet, restoring pods from the older ReplicaSet. Verify the state of ReplicaSets before and after a rollback with:

```
```bash theme={null} kubectl get replicaset
```

For example, before the rollback you might see:

```
```bash theme={null}
```

NAME	DESIRED	CURRENT	READY	AGE
myapp-deployment-67c749c58c	0	0	0	22m
myapp-deployment-7d57dbd8d	5	5	5	20m

After executing the rollback command, the old ReplicaSet is restored while the new one is scaled down.

Summary of Commands

Below is a quick reference table of the key commands discussed in this article:

Command Description	Command
Create the deployment	<code>kubectl create -f deployment-definition.yml</code>
List existing deployments	<code>kubectl get deployments</code>
Update deployment from the YAML definition	<code>kubectl apply -f deployment-definition.yml</code>
Update the container image with kubectl set image	<code>kubectl set image deployment/myapp-deployment nginx-container=nginx:1.9.1</code>
Check the status of the rollout	<code>kubectl rollout status deployment/myapp-deployment</code>
Rollback to a previous deployment revision	<code>kubectl rollout undo deployment/myapp-deployment</code>

With these commands and strategies, you can manage your Kubernetes deployments confidently, ensuring minimal downtime and a reliable process for both updates and rollbacks.

For additional information, see the [Kubernetes Documentation](#).

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/lms.txt> Use this file to discover all available pages before exploring further.

Commands and Arguments in Docker

This article explains Docker commands, arguments, and ENTRYPOINT, focusing on container processes and their relation to Kubernetes pod definitions.

Hello and welcome to this comprehensive lesson on Docker commands, arguments, and entry points. My name is Mumshad Mannambeth, and in this session, we will dive into how containers run processes and how these concepts later translate into pod definitions in Kubernetes. Although these topics are often overlooked in certification curricula, understanding them is essential for mastering containerization.

Understanding Container Commands

When you run a Docker container using the Ubuntu image, as shown below:

```
```bash theme={null} docker run ubuntu
```

Docker launches a container based on the Ubuntu image, which starts and immediately exits since it runs a

```
```bash theme={null}
docker ps
```

you will notice that the container is absent because it has already exited. However, viewing all containers—including those that have stopped—with:

```
```bash theme={null} docker ps -a
```

reveals that the container is in an "exited" state:

```
```plaintext theme={null}
CONTAINER ID      IMAGE      COMMAND      CREATED      STATUS
45aacca36850      ubuntu    "/bin/bash"  43 seconds ago  Exited (0) 41 seconds ago
```

This behavior is different from traditional virtual machines. Containers are optimized to run a single task or process, such as hosting a web server, application server, database, or performing a specific computation. Once the task completes, the container stops because its lifecycle is tied directly to the process running inside it.

Default Commands in Docker Images

Each Docker image contains instructions that define the process to run when a container starts. Many popular images, such as [nginx](#) or [MySQL](#), include a CMD instruction in their Dockerfile that sets the default command. For instance, the nginx image typically has the command `nginx`, and the MySQL image uses `mysqld`.

Consider this Dockerfile snippet for installing and configuring nginx:

```
```dockerfile theme={null}
```

## Install Nginx.

---

```
RUN \ add-apt-repository -y ppa:nginx/stable && \ apt-get update && \ apt-get install -y nginx && \ rm -rf /var/lib/
apt/lists/* && \ echo "\ndaemon off;" >> /etc/nginx/nginx.conf && \ chown -R www-data:www-data /var/lib/nginx
```

## Define mountable directories.

---

```
VOLUME ["/etc/nginx/sites-enabled", "/etc/nginx/certs", "/etc/nginx/conf"]
```

## Define working directory.

---

```
WORKDIR /etc/nginx
```

## Define default command.

---

```
CMD ["nginx"]
```

Now, let's examine the Ubuntu image's Dockerfile. Notice that in this example, the default command is set.

```
``dockerfile theme={null}
Pull base image.
FROM ubuntu:14.04

Install necessary packages.
RUN \
 sed -i 's/# \(*multiverse$\)/\1/g' /etc/apt/sources.list && \
 apt-get update && \
 apt-get -y upgrade && \
 apt-get install -y build-essential software-properties-common byobu curl git htop man unzip vim wget \
 rm -rf /var/lib/apt/lists/*

Add configuration files.
ADD root/.bashrc /root/.bashrc
ADD root/.gitconfig /root/.gitconfig
ADD root/.scripts /root/.scripts

Set environment variables.
ENV HOME /root

Define working directory.
WORKDIR /root

Define default command.
CMD ["bash"]
```

Remember: Bash is a shell, not a persistent server process. When the Ubuntu container is launched without an attached terminal, the shell exits immediately.

## Overriding the Default Command

To override the default command for a container, you can append a command to the end of the `docker run` command. For example, this command instructs the container to run `sleep 5`:

```
``bash theme={null} docker run ubuntu sleep 5
```

In this scenario, the container executes `sleep 5`, waits for five seconds, and then exits.

If you want to permanently change the behavior of the image so that it always runs `sleep 5`, you must create a new Dockerfile.

1. Shell form:

```
``dockerfile theme={null}
FROM ubuntu
CMD sleep 5
```
```

2. JSON array format:

```
``dockerfile theme={null}
FROM ubuntu
CMD ["sleep", "5"]
```
```

Build the new image with:

```
``bash theme={null}
docker build -t ubuntu-sleeper .
```

Then run the container using:

```
``bash theme={null} docker run ubuntu-sleeper
```

By default, this container will sleep for five seconds.

### ## Configuring ENTRYPOINT for Runtime Arguments

Sometimes, you may want to specify only runtime arguments without changing the default command. In such cases, you can use the `ENTRYPOINT` instruction.

Consider the following Dockerfile:

```
``dockerfile theme={null}
FROM ubuntu
ENTRYPOINT ["sleep"]
CMD ["5"]
```

After building and running this image:

```
``bash theme={null} docker build -t ubuntu-sleeper . docker run ubuntu-sleeper
```

the container executes `sleep 5` by default. You can override the sleep duration at runtime by specifying a different argument.

```
``bash theme={null}
docker run ubuntu-sleeper 10
```

This command runs `sleep 10`.

\* With `CMD` alone, runtime arguments replace the default command. \* With `ENTRYPOINT`, runtime arguments are appended to the specified executable, allowing you to override just the parameters.

## Overriding ENTRYPOINT at Runtime

At times, you might want to completely override the ENTRYPOINT. For example, if you wish to use a different command (like switching from `sleep` to `sleep2.0`), you can do so using the `--entrypoint` flag in the `docker run` command.

Given the Dockerfile:

```
```dockerfile theme={null} FROM ubuntu ENTRYPOINT ["sleep"] CMD ["5"]
```

Build the image:

```
```bash theme={null}
docker build -t ubuntu-sleeper .
```

Running the container without modifications:

```
```bash theme={null} docker run ubuntu-sleeper
```

will produce an error because `sleep` expects an operand:

```
```bash theme={null}
docker run ubuntu-sleeper
Output:
sleep: missing operand
Try 'sleep --help' for more information.
```

However, running with a new parameter:

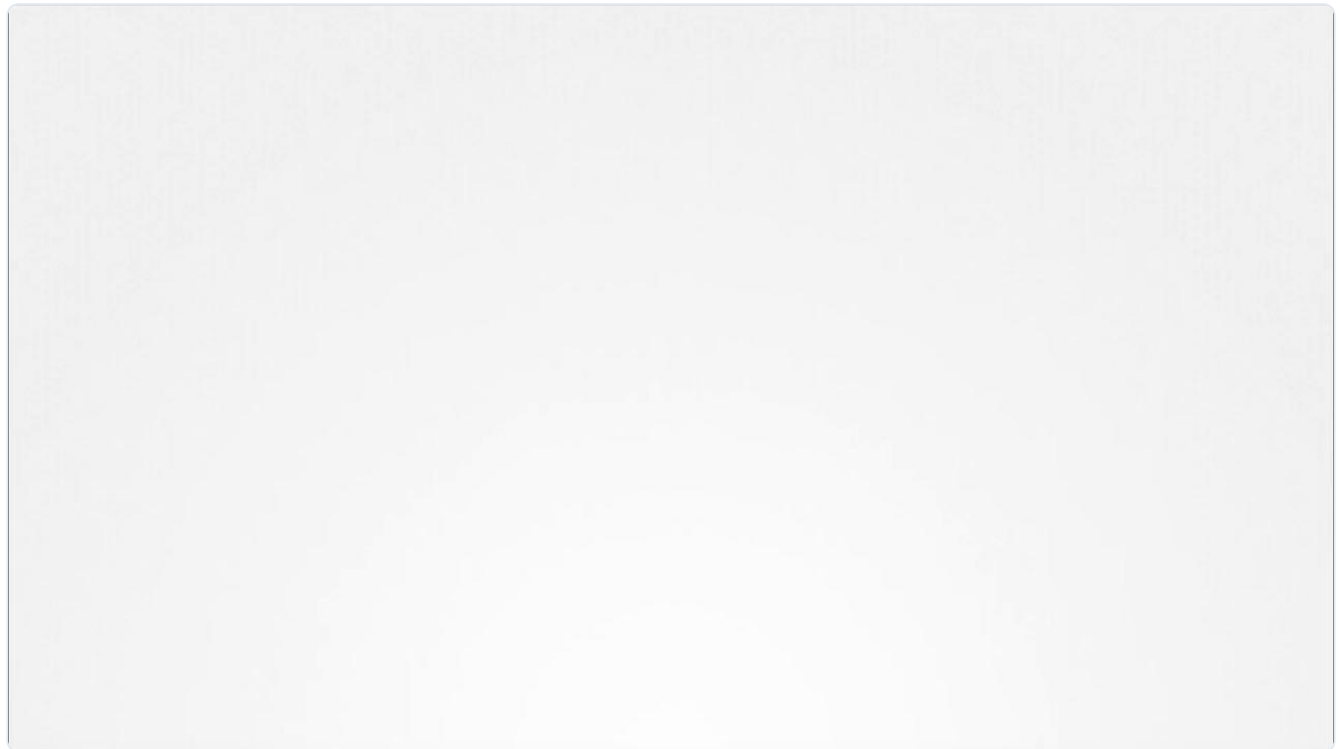
```
```bash theme={null} docker run ubuntu-sleeper 10
```

executes `sleep 10`.

To override the ENTRYPOINT with a different executable, run:

```
```bash theme={null}
docker run --entrypoint sleep2.0 ubuntu-sleeper 10
```

This command starts the container with `sleep2.0 10` (provided that `sleep2.0` is a valid command).



## Conclusion

---

This lesson explained the inner workings of Docker commands, arguments, and how the ENTRYPOINT instruction allows dynamic parameter passing at runtime. These concepts not only help in managing Docker containers efficiently but also lay the groundwork for understanding how they translate into pod definitions in Kubernetes in subsequent lessons.

Happy Dockering!

For further reading, check out the following resources:

- [Docker Documentation](#)
- [Kubernetes Basics](#)
- [Nginx Official Website](#)
- [MySQL Official Website](#)

## Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

# Commands and Arguments in Kubernetes

This lesson covers configuring commands and arguments in Kubernetes pods to adjust container behaviors by overriding default settings from the Dockerfile.

Welcome to this lesson on configuring commands and arguments in Kubernetes pods. In this session, we'll learn how to adjust container behaviors by overriding default settings defined in the Dockerfile via the pod definition.

Previously, we built a simple Docker image—named "ubuntu-sleeper"—that executes a sleep command for a specified number of seconds. By default, running a container with this image makes it sleep for five seconds. However, you can easily change this behavior by passing a command-line argument.

## Overriding Default Behavior with Arguments

Suppose we want to create a pod using the "ubuntu-sleeper" image. We begin with a basic pod definition where the pod's name and image are specified. When the pod is created, it starts a container that runs the default sleep command (sleeping for five seconds) and then exits. To modify the sleep duration to 10 seconds, simply append an additional argument in the pod specification. Any argument provided in the Docker run command correlates with the `args` property in the pod definition file (formatted as an array).

When you append an argument to the Docker run command, it overrides the default parameters defined by the CMD instruction in the Dockerfile.

Consider the following examples:

Docker commands:

```
```bash theme={null} docker run --name ubuntu-sleeper ubuntu-sleeper docker run --name ubuntu-sleeper ubuntu-sleeper 10
```

Pod definition YAML:

```
```yaml theme={null}
apiVersion: v1
kind: Pod
metadata:
 name: ubuntu-sleeper-pod
spec:
 containers:
 - name: ubuntu-sleeper
 image: ubuntu-sleeper
 args: ["10"]
```

## Dockerfile Instructions and Their Mappings

The Dockerfile for the "ubuntu-sleeper" image is defined with both an ENTRYPOINT and a CMD:

```
```dockerfile theme={null} FROM Ubuntu ENTRYPOINT ["sleep"] CMD ["5"]
```

- * The **ENTRYPOINT** instruction specifies the command to run when the container starts.
- * The **CMD** instruction provides default parameters for that command.

By specifying the `args` field in the pod definition file, the CMD instruction is overridden, which effects

Overriding the ENTRYPOINT

Now, consider a scenario where you want to override the ENTRYPOINT itself (for example, switching from "s

Docker command example with overridden ENTRYPOINT:

```
```bash theme={null}
docker run --name ubuntu-sleeper \
 --entrypoint sleep2.0 \
 ubuntu-sleeper 10
```

Pod definition YAML with overridden ENTRYPOINT:

```
```yaml theme={null} apiVersion: v1 kind: Pod metadata: name: ubuntu-sleeper-pod spec: containers: - name:
ubuntu-sleeper image: ubuntu-sleeper command: ["sleep2.0"] args: ["10"]
```

Deployment of the Pod

Once your pod definition is ready, deploy the pod using the following command:

```
```bash theme={null}
kubectl create -f pod-definition.yml
```

## Key Fields Summary

Below is a table summarizing the Kubernetes pod definition fields that map to Dockerfile instructions:

Pod Definition Field	Dockerfile Instruction	Functionality Description
command	ENTRYPOINT	Specifies the command to run when the container starts (completely replacing Dockerfile ENTRYPOINT).
args	CMD	Provides default parameters passed to the command (overriding the Dockerfile CMD).

Remember that specifying the `command` in a pod definition replaces the Dockerfile's ENTRYPOINT entirely, while the `args` field only overrides the default parameters defined by CMD.

This concludes the lesson on managing commands and arguments within your Kubernetes pods. Next, proceed to the coding exercises section to practice these concepts and refine your troubleshooting skills with real-world scenarios.

Happy coding!

## Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/lms.txt> Use this file to discover all available pages before exploring further.

# Secrets

This article explains how to securely manage sensitive data in Kubernetes using Secrets while avoiding common security pitfalls.

Welcome to this comprehensive guide on managing Secrets in Kubernetes. In this article, we explain how to securely handle sensitive data (such as passwords and keys) in your Kubernetes deployments while avoiding common pitfalls like hardcoding credentials in your application.

## Problem with Hardcoding Sensitive Data

Consider a simple Python web application connecting to a MySQL database. When the connection succeeds, the application displays a success message. However, the code includes hardcoded values for hostname, username, and password, which poses a serious security risk.

Previously, configuration data like these values might have been stored in a ConfigMap. For example:

```
```python theme={null} import os from flask import Flask, render_template import mysql.connector

app = Flask(name)

@app.route("/") def main(): mysql.connector.connect(host="mysql", database="mysql", user="root",
password="paswrn") return render_template('hello.html', color=fetchcolor())

if name == "main": app.run(host="0.0.0.0", port="8080")
```

```
```yaml theme={null}
apiVersion: v1
kind: ConfigMap
metadata:
 name: app-config
data:
 # Configuration data goes here
```

While storing non-sensitive details like hostnames or usernames in a ConfigMap is acceptable, placing a password in such a resource is not secure. Kubernetes Secrets provide a mechanism to safely store sensitive information by encoding the data (note: this is not encryption by default).

Secrets encode data using Base64. Although it provides obfuscation, it is not a substitute for encryption.

## Understanding Kubernetes Secrets

Working with Secrets in Kubernetes involves two main steps:

1. **Create the Secret.**
2. **Inject it into a Pod.**

Below is an illustration of Secret data in their encoded and decoded forms:

### Encoded Values

```
```plaintext theme={null} DB_Host: bXlzcWw= DB_User: cm9vdA== DB_Password: cGFzd3Jk
```

```
### Decoded Values
```

```
```plaintext theme={null}
DB Host: mysql
DB User: root
DB Password: paswr
```

There are two primary approaches to creating a Secret:

- **Imperative Creation:** Using the command line to create Secrets on the fly.
- **Declarative Creation:** Defining Secrets in YAML files.

## Imperative Creation of a Secret

With the imperative method, you can supply key-value pairs directly via the command line. For example, to create a Secret named "app-secret" with the key-value pair `DB_Host=mysql`:

```
```bash theme={null} kubectl create secret generic app-secret --from-literal=DB_Host=mysql
```

To include multiple key-value pairs, use the `--from-literal` option repeatedly:

```
```bash theme={null}
kubectl create secret generic app-secret \
 --from-literal=DB_Host=mysql \
 --from-literal=DB_User=root \
 --from-literal=DB_Password=paswd
```

Alternatively, create a Secret from a file with the `--from-file` option:

```
```bash theme={null} kubectl create secret generic app-secret --from-file=app_secret.properties
```

Declarative Creation of a Secret

For a more manageable approach, define a Secret in a YAML file. This file should include the API version,

```
```yaml theme={null}
apiVersion: v1
kind: Secret
metadata:
 name: app-secret
data:
 DB_Host: bXlzcWw=
 DB_User: cm9vdA==
 DB_Password: cGFzd3Jk
```

Apply the definition with the following command:

```
```bash theme={null} kubectl create -f secret-data.yaml
```

Converting Plaintext to Base64

On Linux hosts, you can convert plaintext values to Base64-encoded strings using the `echo -n` command piped to `base64`.

```
```bash theme={null}
echo -n 'mysql' | base64
echo -n 'root' | base64
echo -n 'paswrд' | base64
Output: cGFzd3Jk
```

## Viewing and Decoding Secrets

After creating a Secret, you can list and inspect it with the following commands:

- **List Secrets:**

```
bash theme={null} kubectl get secrets
```

Expected output:

```
plaintext theme={null} NAME TYPE DATA AGE app-secret Opaque 3 10m
```

- **Describe a Secret (without showing sensitive data):**

```
bash theme={null} kubectl describe secret app-secret
```

- **View the encoded data in YAML format:**

```
bash theme={null} kubectl get secret app-secret -o yaml
```

If you need to decode an encoded value, use the `base64 --decode` command:

```
```bash theme={null} echo -n 'bXlzcWw=' | base64 --decode echo -n 'cm9vdA==' | base64 --decode echo -n 'cGFzd3Jk' | base64 --decode
```

Output: paswr

```
## Injecting Secrets into a Pod
```

Once the Secret is created, you can inject it into a Pod using environment variables or by mounting the

```
### Injecting as Environment Variables
```

Below is an example Pod definition that injects the Secret as environment variables:

```
```yaml theme={null}
pod-definition.yaml
apiVersion: v1
kind: Pod
metadata:
 name: simple-webapp-color
 labels:
 name: simple-webapp-color
spec:
 containers:
 - name: simple-webapp-color
 image: simple-webapp-color
 ports:
 - containerPort: 8080
 envFrom:
 - secretRef:
 name: app-secret
```

## Mounting Secrets as Files

Alternatively, mount the Secret as files within a volume. Each key in the Secret becomes a separate file:

```
```yaml theme={null} volumes: - name: app-secret-volume secret: secretName: app-secret
```

After mounting, listing the directory contents should display each key as a file:

```
```bash theme={null}
ls /opt/app-secret-volumes
Output: DB_Host DB_Password DB_User
```

To view the content of a specific file, such as the DB password:

```
```bash theme={null} cat /opt/app-secret-volumes/DB_Password
```

Output: paswr

```
...
```

Important Considerations When Using Secrets

Remember that Kubernetes Secrets are only encoded in Base64, not encrypted by default. Anyone with sufficient access can decode the data. Always handle secret definition files with care and avoid storing them in public repositories.

Here are some key considerations:

- Secrets offer only Base64 encoding. For enhanced security, consider enabling encryption at rest for etcd.
- Limit access to Secrets using Role-Based Access Control (RBAC). Restrict permissions to only those who require it.
- Avoid storing sensitive secret definition files in source control systems that are publicly accessible.
- For even greater security, explore third-party secret management solutions such as AWS Secrets Manager, Azure Key Vault, GCP Secret Manager, or Vault.

External Secret Providers

External secret providers decouple secret management from etcd and offer advanced encryption, granular access control, and comprehensive auditing capabilities. For further details and best practices, consider exploring courses like the [Certified Kubernetes Security Specialist \(CKS\)](#).

Note on Secrets

- ✗ Secrets are not Encrypted. Only encoded.
 - ✗ Do not check-in Secret objects to SCM along with code.
- ✗ Secrets are not encrypted in ETCD
 - ✓ Enable encryption at rest
- ✗ Anyone able to create pods/deployments in the same namespace can access the secrets
 - ✓ Configure least-privilege access to Secrets - RBAC
- ✓ Consider third-party secrets store providers
 - AWS Provider, Azure Provider, GCP Provider, Vault Provider

Conclusion

Managing Kubernetes Secrets effectively is crucial for maintaining the security of your applications. By following the best practices outlined above, including using Secrets to handle sensitive data and applying strict RBAC policies, you can mitigate potential security risks associated with managing sensitive configuration data.

Practice these approaches using hands-on labs and ensure your Kubernetes clusters are secure.

For additional resources, consider the following links:

- [Kubernetes Documentation](#)
- [Kubernetes Basics](#)
- [Docker Hub](#)
- [Terraform Registry](#)

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

Demo Encrypting Secret Data at Rest

This guide explains how to secure secret data in Kubernetes by enabling encryption at rest.

In this guide, we explain how to secure secret data in your Kubernetes cluster by enabling encryption at rest. We start by creating secret objects, examine how Kubernetes encodes them in etcd, and then show you how to configure the API server to encrypt these secrets.

Creating a Secret Object

Begin by launching your Kubernetes playground—a single-node cluster running Kubernetes with ContainerD. Kubernetes makes it easy to create secrets from files, literal values, or environment files. Below are some example commands:

```
```bash theme={null}
```

## Create a secret from all files within a directory:

```
kubectl create secret generic my-secret --from-file=path/to/bar
```

## Create a secret using specified keys from files:

```
kubectl create secret generic my-secret --from-file=ssh-privatekey=path/to/id_rsa --from-file=ssh-publickey=path/to/id_rsa.pub
```

## Create a secret from literal key-value pairs:

```
kubectl create secret generic my-secret --from-literal=key1=supersecret --from-literal=key2=topsecret
```

## Create a secret combining a file and a literal:

```
kubectl create secret generic my-secret --from-file=ssh-privatekey=path/to/id_rsa --from-literal=passphrase=topsecret
```

# Create a secret from environment files:

```
kubectl create secret generic my-secret --from-env-file=path/to/foo.env --from-env-file=path/to/bar.env
```

Additional options like `--allow-missing-template-keys`, `--append-hash`, and `--dry-run` can further refine the command.

After the command executes, verify the secret:

```
```bash theme={null}
kubectl create secret generic my-secret --from-literal=key1=supersecret
kubectl get secret my-secret
```

Using the `describe` command provides detailed metadata, including the base64-encoded data:

```
```bash theme={null} kubectl describe secret my-secret
```

**Secret values are base64-encoded by default; they are not encrypted. Avoid pushing secret configuration to source control.**

\*\*\*

## ## Viewing the Encoded Secret

Kubernetes stores secret values in base64-encoded format. Retrieve the secret as YAML to inspect its contents.

```
```bash theme={null}
kubectl get secret my-secret -o yaml
```

The output might look like:

```
```yaml theme={null} apiVersion: v1 data: key1: c3VwZXJzWmNyZVQ= kind: Secret metadata:
creationTimestamp: "2022-10-24T05:34:13Z" name: my-secret namespace: default resourceVersion: "2111" uid:
dfe97c62-5aa1-46a8-b71c-ffa0cd4c08ec type: Opaque
```

To decode the secret value:

```
```bash theme={null}
echo "c3VwZXJzWmNyZVQ=" | base64 --decode
```

This reveals that the stored secret is only encoded, not encrypted, making it potentially accessible to anyone with access to the YAML output or an etcd dump.

Inspecting Secret Data in etcd

etcd is the key-value store where Kubernetes persists cluster data. Without encryption at rest, secret values remain only base64-encoded, allowing anyone with access to etcd to decode them. Use the `etcdctl` client (API version 3) to query etcd:

```
```bash theme={null} ETCDCTL_API=3 etcdctl \ --cacert=/etc/kubernetes/pki/etcd/ca.crt \ --cert=/etc/kubernetes/pki/etcd/server.crt \ --key=/etc/kubernetes/pki/etcd/server.key \ get /registry/secrets/default/my-secret | hexdump -C
```

Before running the above command, ensure that the `etcdctl` client is installed. On Ubuntu, install it using:

```
```bash theme={null}
apt-get install etcd-client
```

Verify the installation:

```
```bash theme={null} etcdctl
```

Also, check that your control plane node can access the necessary certificate files:

```
```bash theme={null}
ls /etc/kubernetes/pki/etcd/ca.crt
```

The hexdump output will display the raw data, illustrating that without encryption, the secret's value is visible within etcd.

Enabling Encryption at Rest

To protect sensitive data stored in etcd, Kubernetes offers an encryption at rest mechanism using an encryption provider configuration. First, verify if encryption is enabled by checking the kube-apiserver process:

```
```bash theme={null} ps -aux | grep kube-api | grep "encryption-provider-config"
```

If no configuration is found, follow these steps:

### 1. Create an Encryption Configuration File

Generate a random 32-byte key (base64-encoded) with:

```
```bash theme={null}
head -c 32 /dev/urandom | base64
```

Next, create a YAML file (e.g., `enc.yaml`) with the following content (replace the sample key with your generated key):

```
```yaml theme={null} apiVersion: apiserver.config.k8s.io/v1 kind: EncryptionConfiguration resources: - resources:
- secrets providers: - aescbc: keys: - name: key1 secret:
y0xTt+U6xgRdNx4nDYYsijOGgRDoUYC+wAwOKeNfPs= # Replace with your generated key - identity: {}
```

Review the file to ensure accuracy:

```
```bash theme={null}
cat enc.yaml
```

2. Mount the Encryption Configuration File into the API Server

Next, incorporate the configuration into the kube-apiserver by performing these steps:

1. Move the encryption configuration file to a secure directory:

```
bash theme={null} mkdir -p /etc/kubernetes/enc mv enc.yaml /etc/kubernetes/enc/
```

1. Modify the kube-apiserver manifest (found at `/etc/kubernetes/manifests/kube-apiserver.yaml`) by adding the `--encryption-provider-config` flag. Include a new volume and volume mount for the `/etc/kubernetes/enc` directory. For example:

```
yml theme={null} spec: containers: - command: - kube-apiserver # ... other flags ... - --
encryption-provider-config=/etc/kubernetes/enc/enc.yaml volumeMounts: # ... other volume
mounts ... - name: enc mountPath: /etc/kubernetes/enc readOnly: true volumes: # ... other
volumes ... - name: enc hostPath: path: /etc/kubernetes/enc type: DirectoryOrCreate
```

After saving the changes, the kube-apiserver will restart and begin using the new encryption configuration.

Verifying Encryption

Once encryption is activated, any new secret you create will be encrypted at rest. Create a new secret:

```
``bash theme={null} kubectl create secret generic my-secret-2 --from-literal=key2=topsecret
```

Verify the secret's creation:

```
``bash theme={null}
kubectl get secret
```

Then inspect etcd to ensure the secret's value is encrypted:

```
``bash theme={null} ETCDCTL_API=3 etcdctl \ --cacert=/etc/kubernetes/pki/etcd/ca.crt \ --cert=/etc/kubernetes/
pki/etcd/server.crt \ --key=/etc/kubernetes/pki/etcd/server.key \ get /registry/secrets/default/my-secret-2 |
hexdump -C
```

You should observe that the secret's data no longer appears in plain text. Remember, secrets created before enabling encryption are not encrypted.

```
``bash theme={null}
kubectl get secret --all-namespaces -o json | kubectl replace -f -
```

Then confirm that the secrets in etcd are now encrypted.

Always update your existing secrets after enabling encryption at rest to ensure full protection of sensitive data.

Conclusion

This guide demonstrated how Kubernetes handles secret data by showing that, by default, secrets are only base64-encoded—not encrypted—in etcd. We then detailed the process for enabling encryption at rest: creating

an encryption configuration file, mounting it into the API server, and verifying that new secrets store their data securely. By following these steps, you can significantly enhance the security posture of your Kubernetes cluster.

Happy encrypting!

Additional Resources

Resource	Description	Example Command / Link
Kubernetes Secrets	Official documentation on secrets	Kubernetes Secrets
etcd Documentation	Information on etcd and its usage	etcd Documentation
Kubernetes API Server	Details on configuring kube-apiserver flags	Kube-apiserver Docs

For further reading on securing your Kubernetes environment, consider consulting the [Kubernetes Documentation](#).

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/lms.txt> Use this file to discover all available pages before exploring further.

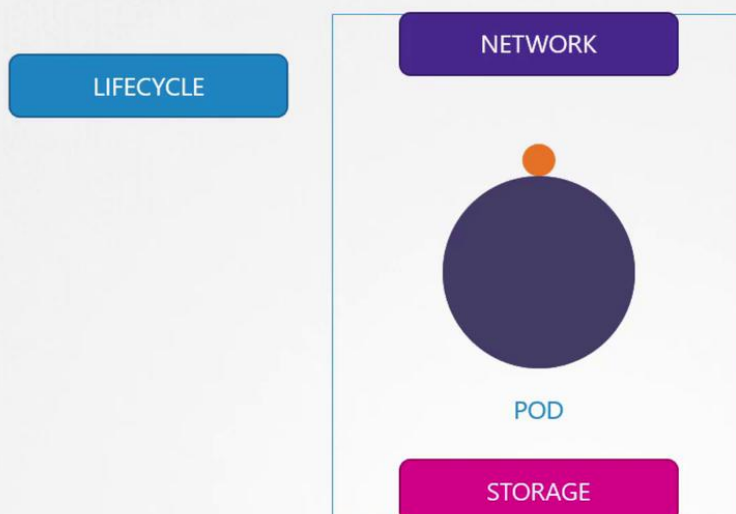
Multi Container Pods

This article explores multi-container pods in Kubernetes, highlighting their benefits for deploying and managing closely linked services together.

In this article, we explore the concept and benefits of multi-container pods in Kubernetes. By breaking down a monolithic application into smaller, independent microservices, you can deploy, manage, and scale each service individually. However, certain scenarios require two closely linked services to run together. For example, a web server might need to be paired with a dedicated logging agent. In this configuration, each web server instance is automatically associated with its own logging service, allowing both services to scale concurrently while keeping their codebases distinct.

Multi-container pods are designed to group containers that share the same lifecycle. This means they are created and terminated together, share a common network namespace (allowing seamless communication via localhost), and have access to shared storage volumes. This design simplifies configurations by eliminating the complexities of volume sharing and networking between separate pods.

Multi-Container PODs



To create a multi-container pod, add the configuration for the new container under the `containers` array in your pod definition file. For instance, you can incorporate a container named "log-agent" alongside an existing web application container. The following YAML snippet demonstrates how to configure a pod that contains both a web application and its corresponding logging agent:

```
yaml theme={null} apiVersion: v1 kind: Pod metadata: name: simple-webapp labels: name: simple-webapp spec: containers: - name: simple-webapp image: simple-webapp ports: - containerPort: 8080 - name: log-agent image: log-agent
```

This configuration ensures that both containers share the same lifecycle, network, and storage resources, allowing them to work together seamlessly.

Remember that the `containers` field is an array in the pod specification, which enables you to define and manage multiple containers under a single pod.

That concludes our discussion on multi-container pods. Next, proceed to the coding exercises section to practice configuring multi-container pods and reinforce your understanding of the concepts discussed here. Happy coding, and see you in the next article!

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

Introduction to Autoscaling 2025 Updates

This article explores autoscaling in Kubernetes, focusing on Horizontal and Vertical Pod Autoscaling for the Certified Kubernetes Administrator exam.

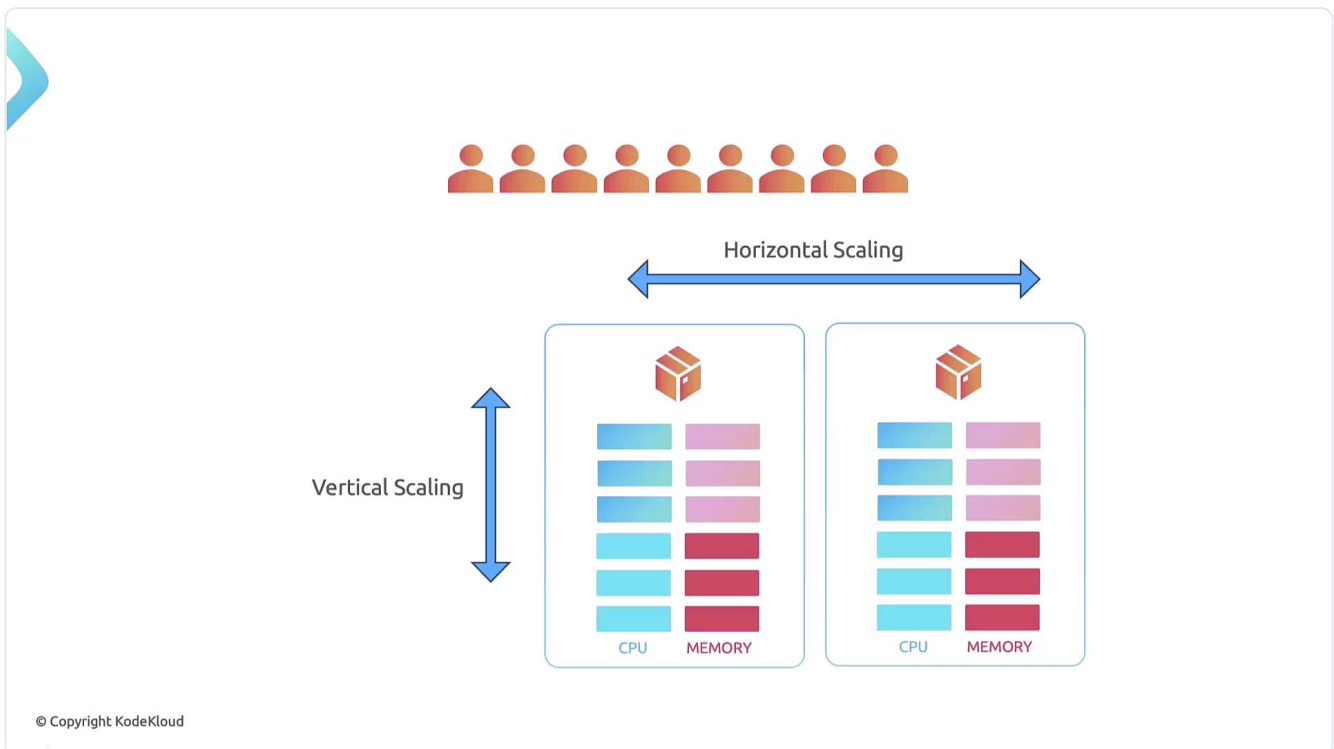
In this lesson series, we explore autoscaling, a key topic for the Certified Kubernetes Administrator (CKA) exam. We will focus on Horizontal Pod Autoscaling (HPA) and Vertical Pod Autoscaling (VPA) to give you the essential knowledge in a concise format. For a comprehensive dive into autoscaling, consider the [Kubernetes Autoscaling course on KodeKloud](#).

Before diving into autoscaling in Kubernetes, let's review the traditional concepts of scaling using physical servers.

Imagine a past scenario where applications were deployed on physical servers with fixed CPU and memory capacities. When the application load exceeded server capacity, you had two options:

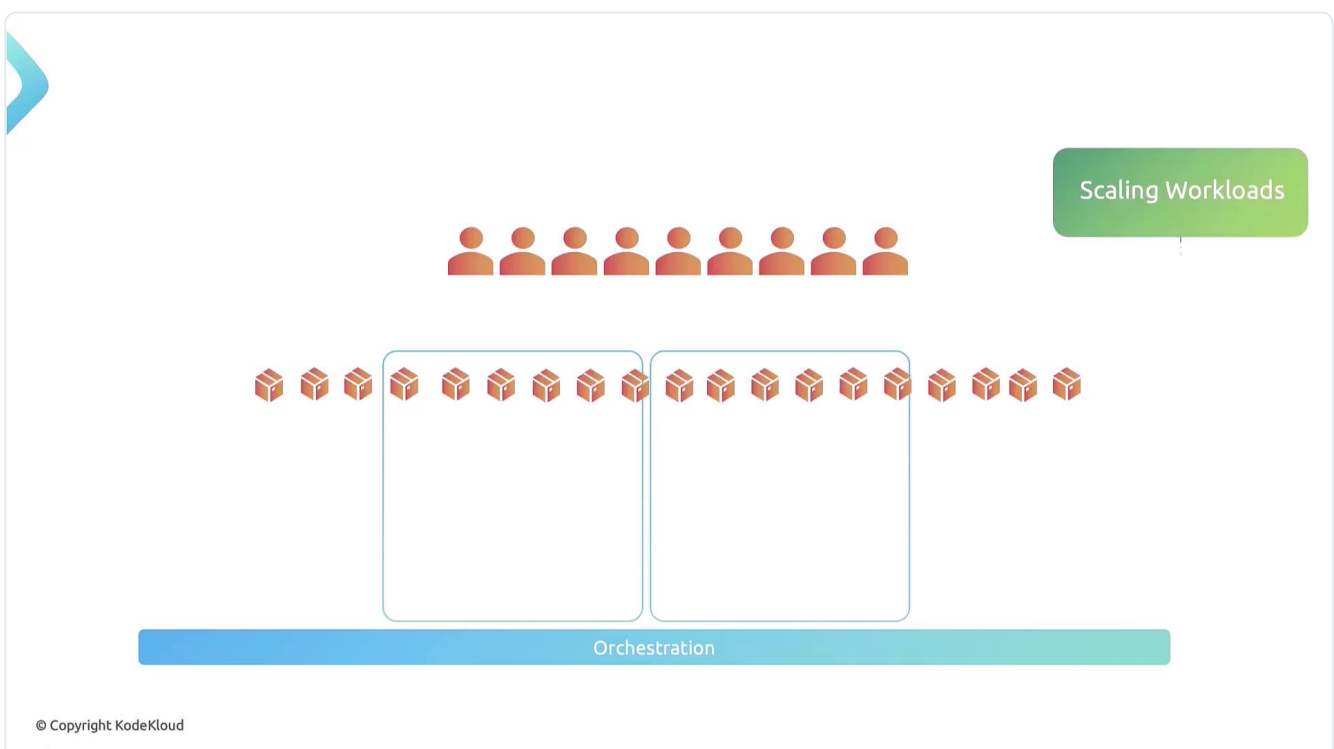
1. Shut down the application and upgrade the existing server by adding more CPU and memory (vertical scaling).
2. If the application could run multiple instances, add another server to distribute the load without downtime (horizontal scaling).

Vertical scaling means enhancing a single server's resources, whereas horizontal scaling means incorporating additional servers to manage increased load.



Now, let's see how these concepts apply to Kubernetes and containerized environments. Kubernetes is designed to dynamically scale containerized applications. Two primary scaling strategies in Kubernetes are:

1. Scaling workloads – adding or removing containers (Pods) in the cluster.
2. Scaling the underlying cluster infrastructure – adding or removing nodes (servers) in the cluster.



To clarify:

- For the cluster infrastructure:

- **Horizontal scaling:** Add more nodes to the cluster.
- **Vertical scaling:** Increase resources (CPU, memory) on existing nodes.
- For workloads:
 - **Horizontal scaling:** Create more Pods.
 - **Vertical scaling:** Increase resource limits and requests for existing Pods.

There are two approaches to scaling in Kubernetes: manual and automated.

Manual scaling and automated scaling both have their place. Manual scaling involves direct intervention and command execution, while automated scaling leverages Kubernetes controllers for dynamic adjustments.

Manual Scaling

For manual scaling, use the following methods:

- **Cluster Infrastructure Horizontal Scaling:** \ Manually provision new nodes and add them to the cluster:

```
bash theme={null} kubeadm join ...
```

- **Workload Horizontal Scaling:** \ Adjust the number of Pods using:

```
bash theme={null} kubectl scale --replicas=<number> <workload-type>/<workload-name>
```

- **Workload Vertical Scaling:** \ Edit the deployment, stateful set, or ReplicaSet to change resource limits and requests:

```
bash theme={null} kubectl edit <workload-type>/<workload-name>
```

Vertical scaling of cluster nodes is less common in Kubernetes because it often requires downtime. In virtualized environments, it may be easier to provision a new VM with higher resources, add it to the cluster, and then decommission the older node.

Automated Scaling

Automated scaling in Kubernetes simplifies operations:

- **Cluster Infrastructure:** \ Managed by the Kubernetes Cluster Autoscaler.
- **Workload Horizontal Scaling:** \ Managed by the Horizontal Pod Autoscaler (HPA).
- **Workload Vertical Scaling:** \ Managed by the Vertical Pod Autoscaler (VPA).

The manual commands summarized earlier are:

```
```bash theme={null} kubeadm join ...
```

```
```bash theme={null}
kubectl scale --replicas=<number> <workload-type>/<workload-name>
```

```
bash theme={null} kubectl edit <workload-type>/<workload-name>
```

This lesson provided a high-level overview of scaling concepts—both in traditional environments and in containerized applications managed by Kubernetes. In upcoming lessons, we will explore these autoscaling methods in greater detail.

See you in the next lesson!

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/lms.txt> Use this file to discover all available pages before exploring further.

Horizontal Pod Autoscaler HPA 2025 Updates

This article explores the Horizontal Pod Autoscaler in Kubernetes and how it automates workload scaling, improving efficiency over manual scaling methods.

In this article, we explore the Horizontal Pod Autoscaler (HPA) feature in Kubernetes and explain how it automates the scaling of workloads. We'll begin by examining the manual approach to scaling an application and then show how HPA streamlines this process.

Manual Horizontal Scaling

As a Kubernetes administrator, you might manually scale your application to ensure it has enough resources during traffic spikes. Consider the following deployment configuration:

```
```yaml theme={null} apiVersion: apps/v1 kind: Deployment metadata: name: my-app spec: replicas: 1 selector: matchLabels: app: my-app template: metadata: labels: app: my-app spec: containers: - name: my-app image: nginx resources: requests: cpu: "250m" limits: cpu: "500m"```
```

In this configuration, each pod requests 250 millicores (mCPU) and is limited to 500 mCPU. To monitor the

```
```bash theme={null}
$ kubectl top pod my-app-pod```
```

The output would be similar to:

```
```bash theme={null} NAME CPU(cores) MEMORY(bytes) my-app-pod 450m 350Mi```
```

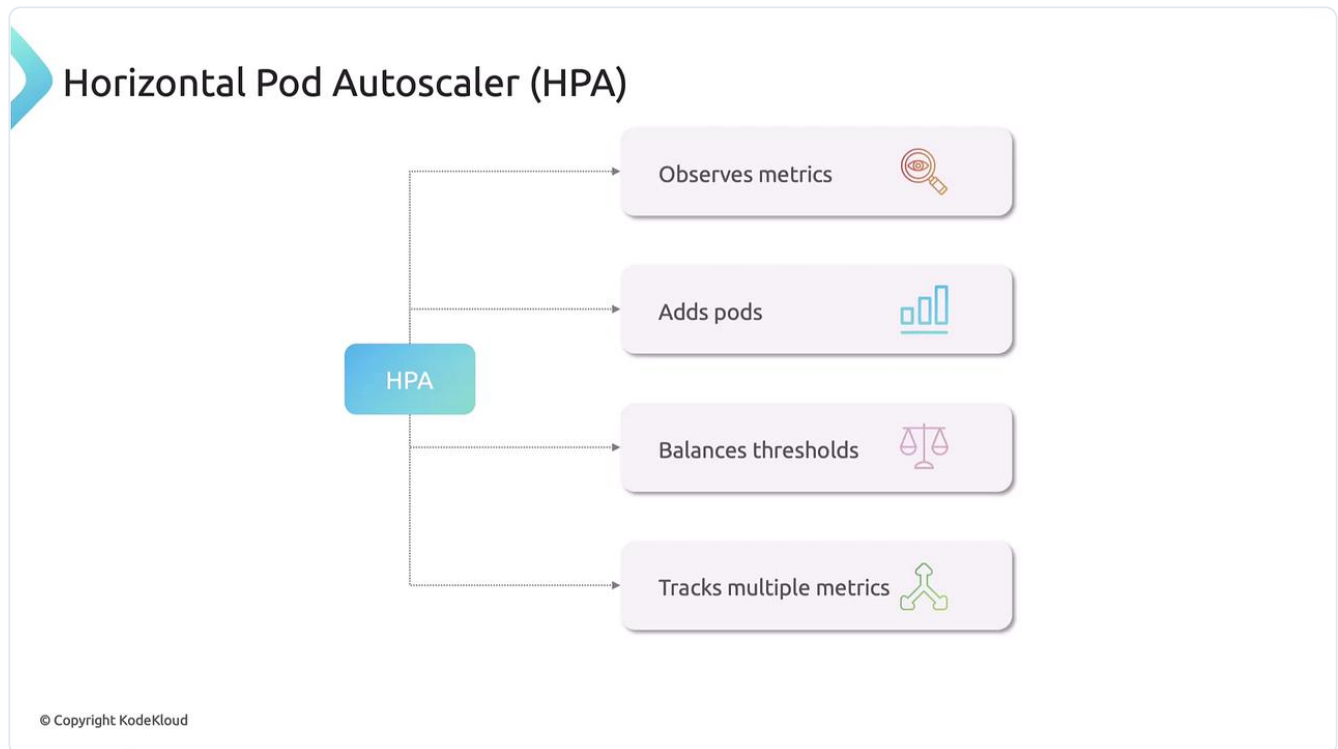
Once you observe the pod's CPU usage nearing the threshold (for example, at 450 mCPU), you would manually

```
```bash theme={null}
$ kubectl scale deployment my-app --replicas=3```
```

Manual scaling requires continuous monitoring and timely intervention, which may not be ideal during unexpected surges in traffic.

Introducing the Horizontal Pod Autoscaler (HPA)

To address the shortcomings of manual scaling, Kubernetes offers the Horizontal Pod Autoscaler (HPA). HPA continuously monitors pod metrics—such as CPU, memory, or custom metrics—using the metrics-server. Based on these metrics, HPA automatically adjusts the number of pod replicas in a deployment, stateful set, or replica set. When resource usage exceeds a preset threshold, HPA increases the pod count; when usage declines, it scales down to conserve resources.



For example, with the nginx deployment above, you can create an HPA by running the command below. This command configures the "my-app" deployment to maintain 50% CPU utilization, scaling the number of pods between 1 and 10:

```
```bash theme={null} $ kubectl autoscale deployment my-app --cpu-percent=50 --min=1 --max=10
```

Kubernetes will then create an HPA that monitors the CPU metrics (using the pod's 500 mCPU limit) via the metrics-server.

To review the status of your HPA, use:

```
```bash theme={null}
$ kubectl get hpa
```

This command shows the current CPU usage, threshold set, and the number of replicas—ensuring that pod counts remain within the defined limits. When the HPA is no longer needed, you can remove it with:

```
```bash theme={null} $ kubectl delete hpa my-app
```

### ## Declarative Configuration for HPA

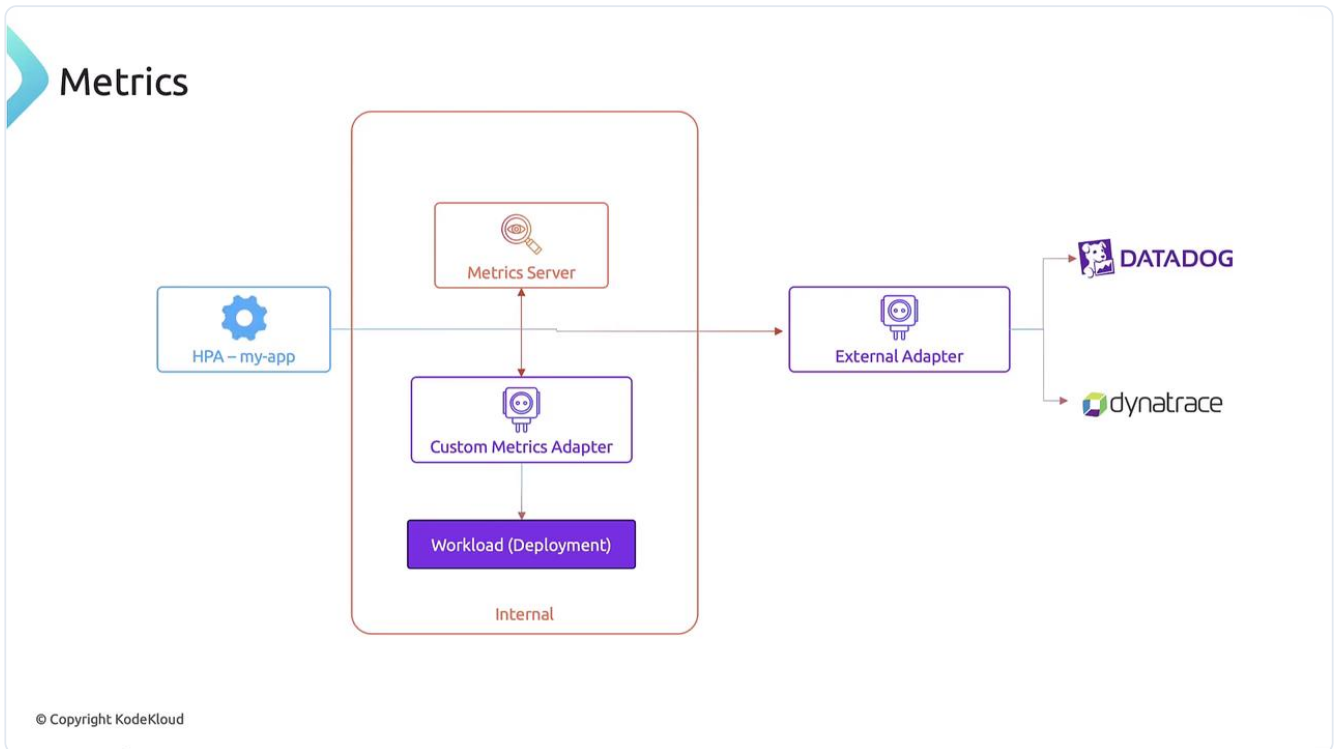
Beyond the imperative approach, you can declare the HPA configuration with a YAML file. Here's an example

```
```yaml theme={null}
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: my-app-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: my-app
  minReplicas: 1
  maxReplicas: 10
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 50
```

This configuration ensures that the HPA monitors the CPU utilization of the "my-app" deployment, automatically adjusting the replica count as needed. Note that HPA, integrated into Kubernetes since version 1.23, relies on the metrics-server to obtain resource utilization data.

Metrics Sources and External Adapters

Kubernetes supports not only the internal metrics-server for collecting CPU or memory metrics but also custom metrics adapters. These adapters can retrieve metrics from other internal sources or external metrics providers like Datadog or Dynatrace through an external adapter. For further details on advanced configurations, please explore our [Kubernetes Autoscaling](#) course.



Conclusion

This article provided a comprehensive overview of the Horizontal Pod Autoscaler (HPA) in Kubernetes. We discussed the drawbacks of manual scaling and demonstrated how HPA automates scaling based on real-time resource usage. Whether through imperative commands or declarative YAML configurations, HPA ensures that your applications can adapt dynamically to fluctuating workloads.

For additional insights and hands-on experience, consider enrolling in our [Kubernetes Autoscaling](#) course.

Happy scaling!

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

In place Resize of Pods 2025 Updates

This guide explores in-place resizing of Pod resources to streamline updates and reduce downtime for stateful workloads without recreating the entire Pod.

In this guide, we explore how to perform in-place resizing of Pod resources—a feature that streamlines updates to resource changes without recreating the entire Pod. This innovative approach reduces downtime, especially for stateful workloads, by updating resource requirements directly on the running Pods.

Understanding the Default Behavior

By default, Kubernetes (v1.32 and later) replaces the existing Pod when you modify resource requests or limits in a Deployment. Consider the following deployment configuration:

```
```yaml
theme={null}
apiVersion: apps/v1
kind: Deployment
metadata:
 name: my-app
spec:
 replicas: 1
selector:
 matchLabels:
 app: my-app
template:
 metadata:
 labels:
 app: my-app
 spec:
 containers:
 - name: my-app
 image: nginx
 resources:
 requests:
 cpu: "250m"
 memory: "256Mi"
 limits:
 cpu: "500m"
 memory: "512Mi"
```

When you update the resource requirements, Kubernetes terminates the existing Pod and creates a new one with the updated resources.

## ## The In-Place Update Mechanism

To address the disruption from full Pod recreation, Kubernetes is developing an in-place update mechanism.

### ### Example of In-Place Update Manifest

Below is an example where the CPU resource is increased from "250m" to "1". Thanks to the in-place update mechanism, the Pod is updated without being recreated.

```
```yaml theme={null}
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-app
spec:
  replicas: 1
  selector:
    matchLabels:
      app: my-app
  template:
    metadata:
      labels:
        app: my-app
    spec:
      containers:
      - name: my-app
        image: nginx
        resources:
          requests:
            cpu: "1"
            memory: "256Mi"
          limits:
            cpu: "500m"
            memory: "512Mi"
```

To activate this feature, enable the in-place Pod vertical scaling feature flag with the following command:

```
```shell theme={null} $ FEATURE_GATES=InPlacePodVerticalScaling=true
```

Once enabled, you can specify additional resize policy parameters to control restart behavior for each resource.

### ### Configuring Resize Policies

The following manifest demonstrates a resize policy for the CPU resource, allowing an update without restart.

```
```yaml theme={null}
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-app
spec:
  replicas: 1
  selector:
    matchLabels:
      app: my-app
  template:
    metadata:
      labels:
        app: my-app
    spec:
      containers:
        - name: my-app
          image: nginx
          resizePolicy:
            - resourceName: cpu
              restartPolicy: NotRequired
      resources:
        requests:
          cpu: "250m"
          memory: "256Mi"
        limits:
          cpu: "500m"
          memory: "512Mi"
```

When the in-place update feature is active, increasing the CPU resource value (for example, from "250m" to "1") updates the running Pod directly without termination, ensuring smooth scalability and minimal service interruption.

The in-place resizing feature currently supports only CPU and memory resources. Familiarize yourself with these constraints before implementation.

Limitations of In-Place Pod Resizing

It is important to note the following limitations with the current implementation of in-place resizing:

- Only CPU and memory resources can be updated in place.
- Changes to Pod QoS classes and certain other attributes are not supported.
- Init containers and ephemeral containers are not eligible for in-place resizing.
- Resource requests and limits, once assigned to a container, cannot be shifted to another container.
- A container's memory limit cannot be reduced below its current usage; if such a request is made, the resize operation will remain in progress until the new memory limit is achievable.

- Windows Pods are not supported by this feature.

Looking Ahead: Vertical Pod Autoscaler

While this article focused on manually resizing Pod resources, Kubernetes also offers the Vertical Pod Autoscaler (VPA) to automate scaling based on resource usage. VPA continuously monitors your application's resource consumption and adjusts requests and limits as needed, further reducing manual intervention.

For more details on Kubernetes resource management, visit the [Kubernetes Documentation](#) or explore [Kubernetes Basics](#).

That's it for this topic. Enjoy seamless in-place updates and improved resource scaling with Kubernetes!

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/lms.txt> Use this file to discover all available pages before exploring further.

Vertical Pod Autoscaling VPA 2025 Updates

This article explores optimizing Kubernetes workloads by using the Vertical Pod Autoscaler to automatically adjust resource allocations for applications.

In this article, we explore how to optimize Kubernetes workloads by scaling them vertically using the Vertical Pod Autoscaler (VPA). As a Kubernetes administrator, your goal is to ensure that applications always receive optimal resource allocations, such as CPU and memory. Let's start by examining a typical deployment configuration for a pod that specifies a CPU request of 250 millicores and a limit of 500 millicores:

```
```yaml theme={null} apiVersion: apps/v1 kind: Deployment metadata: name: my-app spec: replicas: 1 selector: matchLabels: app: my-app template: metadata: labels: app: my-app spec: containers: - name: my-app image: nginx resources: requests: cpu: "250m" limits: cpu: "500m"```
```

In this setup, the pod cannot use more than 500 millicores of CPU. To monitor its resource consumption, ex

```
```bash theme={null}
$ kubectl top pod my-app-pod
NAME          CPU(cores)   MEMORY(bytes)
my-app-pod    450m        350Mi```
```

If the pod's CPU consumption reaches a predefined threshold, you might need to update its resource specifications manually. For example, you can increase the CPU request to "1" while keeping the limit unchanged:

```
```yaml theme={null} apiVersion: apps/v1 kind: Deployment metadata: name: my-app spec: replicas: 1 selector: matchLabels: app: my-app template: metadata: labels: app: my-app spec: containers: - name: my-app image: nginx resources: requests: cpu: "1" limits: cpu: "500m"```
```

To apply this change, run:

```
```bash theme={null}
$ kubectl edit deployment my-app```
```

After saving, Kubernetes will terminate the current pod and create a new one with the updated resource configuration.

Manually updating pods can be time-consuming and error-prone. Kubernetes provides the Vertical Pod Autoscaler (VPA) to automate this process.

Kubernetes distinguishes between scaling methods. While the Horizontal Pod Autoscaler (HPA) adds or removes pods based on demand, the VPA continuously monitors metrics and automatically adjusts the CPU and memory allocation of each pod. Since VPA is not enabled by default, you must install it manually. Start by applying the VPA definition file from the autoscaler GitHub repository:

```
```bash theme={null} $ kubectl apply -f https://github.com/kubernetes/autoscaler/releases/latest/download/vertical-pod-autoscaler.yaml
```

Verify that the VPA components are running in the kube-system namespace:

```
```bash theme={null}
$ kubectl get pods -n kube-system | grep vpa
vpa-admission-controller-xxxx    Running
vpa-recommender-xxxx            Running
vpa-updater-xxxx                Running
```

The VPA deployment includes three key components:

1. **VPA Recommender:** Continuously monitors resource usage via the Kubernetes metrics API, analyzes historical and live data, and provides optimized recommendations for CPU and memory.
2. **VPA Updater:** Compares current pod resource settings against recommendations and evicts pods running with suboptimal resources. This eviction triggers the creation of new pods with updated configurations.
3. **VPA Admission Controller:** Intercepts pod creation requests and mutates the pod specification based on the recommender's suggestions, ensuring that new pods start with the ideal resource configuration.

Next, create a VPA resource with a YAML definition. Unlike HPA, the VPA isn't set up through imperative commands. The example below shows a configuration that monitors the "my-app" deployment, enforces minimum and maximum CPU limits, and uses the "Auto" update mode:

```
```yaml theme={null} apiVersion: autoscaling.k8s.io/v1 kind: VerticalPodAutoscaler metadata: name: my-app-vpa
spec: targetRef: apiVersion: apps/v1 kind: Deployment name: my-app updatePolicy: updateMode: "Auto"
resourcePolicy: containerPolicies: - containerName: "my-app" minAllowed: cpu: "250m" maxAllowed: cpu: "2"
controlledResources: ["cpu"]
```

In "Auto" mode, the VPA updater behaves similarly to a "recreate" strategy by terminating pods that run with

To inspect the resource recommendations provided by VPA for your deployment, run:

```
```bash theme={null}
$ kubectl describe vpa my-app-vpa
```

You might see an output similar to this, which indicates a recommended CPU value of 1.5:

```
text theme={null} Recommendations: Target: Cpu: 1.5
```

Comparing Vertical and Horizontal Pod Autoscaling

Understanding when to use VPA versus HPA is crucial for efficient resource management:

Feature	Vertical Pod Autoscaling (VPA)	Horizontal Pod Autoscaling (HPA)
Scaling Method	Adjusts CPU and memory settings of individual pods (may restart pods for changes).	Increases or decreases the number of pods to distribute load.
Pod Behavior	May cause temporary downtime during pod restarts.	Scales pods seamlessly without interrupting existing ones.
Traffic Handling	Less effective for sudden spikes due to restart delays.	Ideal for handling rapid traffic spikes by adding more pods instantly.
Cost Optimization	Prevents over-provisioning by matching resource allocation with actual usage.	Reduces operational costs by avoiding underutilized pods.
Use Cases	Stateful workloads, databases, JVM-based applications, and AI workloads requiring precise tuning.	Stateless applications, web services, and microservices requiring rapid scaling.

Key Differences

Feature	VPA (Vertical Scaling)	HPA (Horizontal Scaling)
Scaling Method	Increases CPU & memory of existing Pods	Adds/removes Pods based on load
Pod Behavior	Restarts Pods to apply new resource values	Keeps existing Pods running
Handles Traffic Spikes?	❌ No, because scaling requires a Pod restart	✅ Yes, instantly adds more Pods
Optimizes Costs?	✅ Prevents over-provisioning of CPU/memory	✅ Avoids unnecessary idle Pods
Best For	Stateful workloads, CPU/memory-heavy apps (DBs, ML workloads)	Web apps, microservices, stateless services
Example Use Cases	Databases (MySQL, PostgreSQL), JVM-based apps, AI/ML workloads	Web servers (Nginx, API services), message queues, microservices

© Copyright KodeKloud

VPA focuses on optimizing resource allocation for individual pods, while HPA scales the number of pods to meet demand. The choice depends on your application's workload characteristics and scaling requirements.

In summary, the Vertical Pod Autoscaler (VPA) enhances Kubernetes resource management by dynamically adjusting CPU and memory allocations based on real-time metrics. By applying these VPA techniques, you can ensure that your applications run efficiently without manual intervention.

Now, try deploying VPA in your Kubernetes environment to experience improved resource optimization and operational efficiency.